

Project Documentation — Ansible Web + Database Deployment

Course: LF10 · Control node: laptop · Target: Raspberry Pi (100.94.232.78)

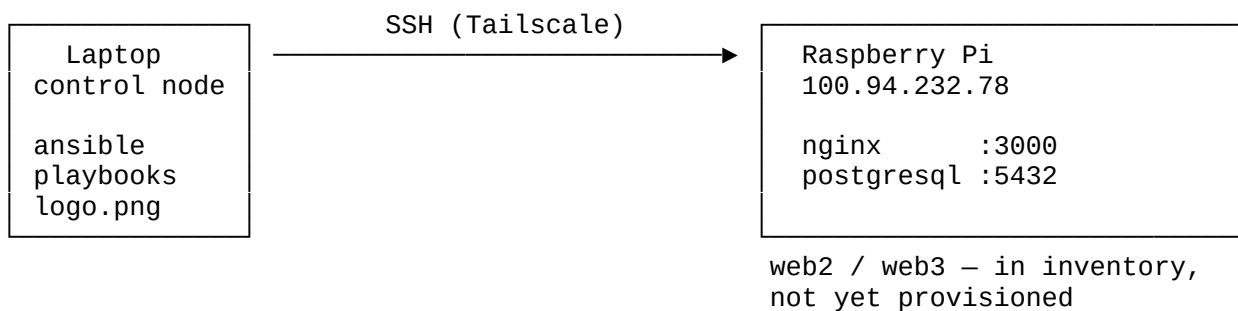
1. Goal

Deploy, fully automated with Ansible:

- an **nginx web server** serving a page that reads *"This server is handled by Ansible"*
- a **PostgreSQL database** holding an image that is displayed on that page
- an inventory describing **3 web servers and 1 database server**

The Raspberry Pi hosts both roles (it is a member of both groups). The other two web hosts exist in the configuration but are not deployed yet — this proves the setup scales without changing any role code.

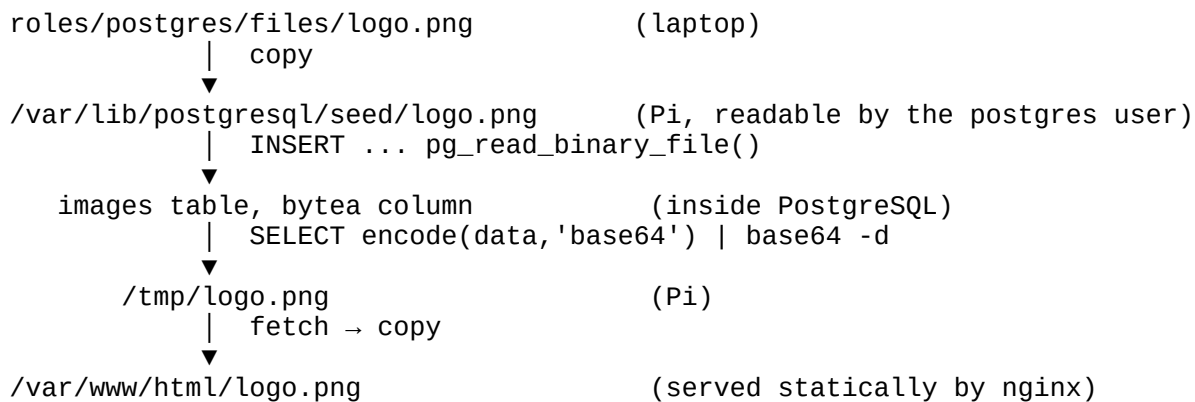
2. Architecture



Ansible is **agentless**: nothing is installed on the Pi to support it. It connects over SSH and only needs Python 3, which Raspberry Pi OS already provides.

3. How the image gets from the database to the browser

This was the central design decision. The image lives in PostgreSQL, but the web server has **no application layer** (no PHP, no Node) to query it at request time. Instead the transfer happens **during the deployment**:



Trade-off: the browser gets a static file, so the page load never touches the database. The image is a *snapshot* taken at deploy time. Changing the image in the database requires re-running the playbook. In exchange, the web tier stays radically simple — nginx and two files.

4. File-by-file reference

```
ansible-basic/
├── ansible.cfg
├── inventory.yml
├── group_vars/all.yml
├── site.yml
├── rollback.yml
├── scripts/check.sh
├── roles/
│   ├── postgres/
│   │   ├── files/logo.png
│   │   └── tasks/main.yml
│   └── nginx/
│       ├── tasks/main.yml
│       ├── handlers/main.yml
│       └── templates/
│           ├── default.j2
│           └── index.html.j2
```

ansible.cfg

Project-level settings, picked up automatically **when you run Ansible from this directory**. Sets the inventory path, the roles path, disables SSH host-key prompts, and enables `become` (sudo) globally so individual tasks don't each need it.

inventory.yml

The list of managed machines, in YAML format. Defines two groups:

Group	Hosts
webserver	jamen (the Pi), web2, web3
dbserver	jamen

The Pi appears in **both groups** under the same alias, which is why one machine receives both roles. `ansible_host` maps the alias to an IP; `ansible_user` sets the SSH login.

Alias vs. user: the alias (`jamen`) is what you type on the command line, `ansible_user` is the SSH account. They happen to share a name here, which caused early confusion — `ansible jamen -m ping` only worked once the *host* was named `jamen`.

group_vars/all.yml

Central variable file. Every value used by more than one place lives here, so ports, names and text are changed in exactly one spot:

Variable	Purpose
db_name	database name (webapp)
db_port	PostgreSQL port — must match <code>pg_lsclusters</code>
image_name	primary key of the row in <code>images</code>
image_file	file name, also determines the MIME type nginx sends
web_root	<code>/var/www/html</code>
web_port	3000
web_message	the required text on the page

site.yml

The main playbook. Two plays, executed in order:

1. `dbserver` → `postgres` role (the image must be in the DB first)
2. `webserver` → `nginx` role (which then reads the image out again)

Order matters — reversing it would break the export step.

roles/postgres/tasks/main.yml

1. install `postgresql`, `python3-psycopg2` (the modules need it), `acl` (needed for `become_user: postgres`)
2. start and enable the service
3. create the `webapp` database
4. create the `images` table (`name text PRIMARY KEY, data bytea`)
5. copy `logo.png` to a directory the `postgres` user can read
6. `INSERT ... ON CONFLICT DO UPDATE` the image into the table

Step 6 uses `pg_read_binary_file()`, which runs **server-side** — this avoids base64-encoding the image through Ansible and works for any file size. The `WHERE ... IS DISTINCT FROM` clause makes it idempotent: re-running only writes when the bytes actually differ.

roles/postgres/files/logo.png

The source image. Any PNG works; it was generated locally as a placeholder.

Ansible's `copy` module looks in a role's `files/` directory by convention, so only the bare file name is given in the task.

roles/nginx/tasks/main.yml

1. install nginx
2. render `default.j2` → `/etc/nginx/sites-available/default` (this is what moves the server to port 3000)
3. ensure the symlink in `sites-enabled/` exists
4. **run nginx -t** to validate the config
5. start and enable nginx
6. render `index.html.j2` → `/var/www/html/index.html`
7. export the image from PostgreSQL to `/tmp`, fetch it to the laptop, copy it into the web root

Step 4 was added after a debugging session: `service: state=started` only reports "*the control process exited with an error code*", which says nothing. `nginx -t` fails with nginx's own message instead, so the cause appears directly in the Ansible output.

The export tasks use `delegate_to: "{{ groups['dbservers'][0] }}"` and `run_once: true` — they run **on the database host**, once, no matter how many web servers are in the play. This is what makes the setup work unchanged for 1 or 100 web servers.

roles/nginx/handlers/main.yml

A handler triggered by `notify:`. It runs **once at the end of the play**, and only if something actually changed — so editing the config and the HTML in one run restarts nginx once, not twice.

Originally this was `reload`. It was changed to `restart` because `reload` fails outright when nginx is stopped — exactly the situation after a failed run, which would have made the playbook impossible to recover with a simple re-run.

roles/nginx/templates/default.j2

Jinja2 template for the nginx server block. `{{ web_port }}` is substituted at deploy time. The IPv6 `listen [::]` line is wrapped in an `{% if %}` because it aborts nginx startup on systems with IPv6 disabled.

roles/nginx/templates/index.html.j2

The page itself. Renders `{{ web_message }}` as the heading and references `{{ image_file }}`. Because it is a template rather than a static file, all three web servers produce **byte-identical output** from the same source.

scripts/check.sh

Test helper. Snapshots the state of the system: HTTP status codes from the laptop, plus package status, running services, listening ports, deployed files, and row count in the database from the Pi. Run before and after a deploy and `diff` the two outputs.

rollback.yml

Undo playbook with two modes:

- **soft** (default) — removes the deployed files, drops the `webapp` database, restores nginx's stock port-80 config
- **purge** (`-e purge=true`) — additionally uninstalls nginx and PostgreSQL and deletes their data directories, restoring a genuinely clean "before" state

5. Ansible concepts demonstrated

Concept	Where
Roles	<code>postgres</code> and <code>nginx</code> — reusable, self-contained units
Inventory groups	one host in two groups → two roles on one machine
Variables	<code>group_vars/all.yml</code> as the single source of truth
Templates (Jinja2)	config and HTML generated from variables
Handlers	service restart only when something changed
Idempotency	<code>state: present, IF NOT EXISTS, IS DISTINCT FROM</code>
Delegation	<code>delegate_to + run_once</code> for the DB export
Privilege escalation	<code>become / become_user: postgres</code>

Idempotency is the property worth highlighting: running the playbook a second time reports `changed=0` because every task first checks whether the desired state already exists. This is the fundamental difference between a configuration management tool and a shell script.

6. Workflow

bash

```
# preparation (once)
ansible-galaxy collection install community.postgresql
ssh-copy-id jamen@100.94.232.78
ansible jamen -m ping

# before/after test
./scripts/check.sh | tee before.txt
ansible-playbook site.yml --limit jamen --ask-become-pass
./scripts/check.sh | tee after.txt
diff -u before.txt after.txt

# prove idempotency
ansible-playbook site.yml --limit jamen --ask-become-pass    # → changed=0

# undo
```

```
ansible-playbook rollback.yml --limit jamen --ask-become-pass
```

--limit jamen restricts the run to the Pi, since web2 and web3 do not exist yet.

Result: <http://100.94.232.78:3000/>

7. Problems encountered

Problem	Cause	Solution
Could not match supplied host pattern: jamen	confused the inventory alias with <code>ansible_user</code>	renamed the host in the inventory (in both groups)
Python interpreter warning	Ansible discovers <code>/usr/bin/python 3.13</code> dynamically	<code>interpreter_python = auto_silent</code> in <code>ansible.cfg</code>
Unable to start service nginx	error message from <code>service:</code> is not diagnostic	added an explicit <code>nginx -t</code> validation task
Port 5432 already in use	another PostgreSQL/container already on the Pi	introduced <code>db_port</code> ; Debian moves a second cluster to 5433 automatically

8. Open points

- **Verify `db_port`.** Run `pg_lsclusters` on the Pi. If the cluster is on 5433, `db_port` must say 5433 — otherwise the DB tasks silently address the wrong server.
 - **Do not use `-e purge=true`** while another PostgreSQL exists on the Pi. `apt-get purge 'postgresql*' plus the removal of /var/lib/postgresql would destroy that database as well. Soft rollback is safe.`
 - **web2 / web3** still carry placeholder IPs. Once real machines exist, drop `--limit` and they receive the identical configuration with no code changes.
-

9. Possible extensions

- a load balancer (HAProxy/nginx) in front of the three web servers
- HTTPS via a Let's Encrypt role
- `ansible-vault` for the database password once network authentication is introduced
- a live PHP endpoint reading the image from PostgreSQL on every request instead of a deploy-time snapshot
- `pg_dump` backups as an additional task in the `postgres` role